FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO

ALGORITMOS E LÓGICA DE PROGRAMAÇÃO

Gabriel Souza Lima
Larissa Destro Sousa
Luan da Silva Santos
Pedro Henrique Luvizotto Uehara

APLICAÇÃO DE ALGORITMOS E LÓGICA DE PROGRAMAÇÃO NO JOGO “PROTOCOLO ARCOR”

São Paulo
2026

Gabriel Souza Lima
Larissa Destro Sousa
Luan da Silva Santos
Pedro Henrique Luvizotto Uehara

APLICAÇÃO DE ALGORITMOS E LÓGICA DE PROGRAMAÇÃO NO JOGO “PROTOCOLO ARCOR”

Trabalho acadêmico apresentado à disciplina de Algoritmos e Lógica de Programação, do curso de Ciência da Computação da Fundação Escola de Comércio Álvares Penteado (FECAP), como parte dos requisitos de avaliação da Entrega 2 do projeto interdisciplinar.

São Paulo
2026

SUMÁRIO

1 INTRODUÇÃO 4
1.1 Objetivo 4
1.2 Delimitação 4
2 O JOGO PROTOCOLO ARCOR 4
3 ORGANIZAÇÃO DOS SCRIPTS 5
3.1 GerenciadorFase.cs 5
3.2 IntroducaoHistoria.cs 6
3.3 Camera.cs 6
3.4 Pegar 6
3.5 PlayerMovement.cs 7
4 ESTRUTURAS DE PROGRAMAÇÃO UTILIZADAS 7
4.1 Procedimentos e parâmetros 7
5 PSEUDOCÓDIGOS DOS PRINCIPAIS ALGORITMOS 9
5.1 Controle do tempo e dicas 9
5.2 Leitura e registro de pistas 10
5.3 Entrada e confirmação do código 10
5.4 Vitória e pontuação 11
5.5 Pegar ou soltar um objeto 12
5.6 Movimentação do personagem 13
5.7 Rotação da câmera 13
5.8 Apresentação das falas 13
6 RESULTADOS 14
6.1 Cenários de verificação 14
6.2 Cálculo da pontuação 15
7 CONCLUSÃO 15
REFERÊNCIAS 17
APÊNDICE A – EVIDÊNCIAS DOS SCRIPTS (CAPTURAS DE TELA) 18

1 INTRODUÇÃO

O presente relatório integra a Entrega 2 da unidade curricular de Algoritmos e Lógica de Programação do projeto interdisciplinar desenvolvido em parceria entre a Fundação Escola de Comércio Álvares Penteado (FECAP) e a Arcor (FECAP, 2026). Ele documenta como algoritmos estruturados em procedimentos foram aplicados nos scripts em C# do jogo “Protocolo Arcor”, desenvolvido na plataforma Unity.

1.1 Objetivo

Demonstrar a aplicação de variáveis, operadores, estruturas condicionais, estruturas de repetição e procedimentos na implementação do jogo, apresentando os algoritmos principais em pseudocódigo e os resultados que eles produzem durante a partida.

1.2 Delimitação

Conforme a orientação da disciplina, este documento se restringe à aplicação dos algoritmos e aos seus resultados no projeto. As decisões do código foram escritas com estruturas if/else explícitas, sem uso do operador ternário. Os trechos de código analisados são reproduzidos, em capturas de tela, no Apêndice A (Figuras 1 a 14), e as referências a eles aparecem ao longo do texto.

2 O JOGO PROTOCOLO ARCOR

O Protocolo Arcor é um jogo tridimensional em primeira pessoa, desenvolvido em Unity para a parceria entre a FECAP e a Arcor. O jogador assume o técnico Fúlvio Marques, que entra em uma fábrica às escuras depois de uma tempestade e precisa reativar as estações antes que o tempo acabe, orientado por rádio pelo Block Man, um funcionário da fábrica.

A introdução apresenta a situação em seis falas: uma tempestade atípica sobrecarrega os geradores, o painel principal solta faíscas e o Protocolo Arcor trava todas as estações. Na Fase 1 (Estação 7 Belo), o jogador tem 120 segundos para explorar o ambiente, encontrar quatro bilhetes com pistas e digitar no painel de

controle o código que reativa a fábrica. Cada dígito depende do anterior, e por isso as pistas precisam ser usadas na ordem correta.

Quando o código está correto, a fase termina em vitória: a pontuação é calculada a partir do tempo restante, o melhor tempo é gravado no computador do jogador e as luzes da fábrica voltam a acender. Quando o tempo chega a zero, a tela de derrota é exibida e a fase pode ser reiniciada. Esse comportamento resulta da cooperação de cinco scripts, descritos na seção 3.

3 ORGANIZAÇÃO DOS SCRIPTS

A solução foi decomposta em scripts com responsabilidades distintas, evitando concentrar todo o comportamento em um único bloco de código. O Quadro 1 resume essa divisão e as subseções seguintes detalham cada script.

Quadro 1 – Scripts do jogo e suas responsabilidades

Script	Responsabilidade	Principais recursos de lógica
GerenciadorFase	Coordena o desafio da fase: cronômetro, dicas do Block Man, registro das pistas, painel numérico, vitória, derrota e reinício.	Variáveis de estado, vetores, if/else, for, procedimentos com parâmetros
IntroducaoHistoria	Conduz a narrativa de abertura: falas com efeito de digitação, movimento da câmera, animação do Block Man e efeitos de luz.	Vetor de falas, if/else, for, números aleatórios
Camera	Lê o mouse e orienta a visão em primeira pessoa.	Operadores aritméticos, Mathf.Clamp
Pegar (arquivo PickupSystem.cs)	Pega e solta objetos usando um raio lançado a partir da câmera.	if/else, Physics.Raycast
PlayerMovement	Desloca o personagem e aplica a gravidade.	Operadores aritméticos, if

Fonte: Elaborado pelos autores (2026).

3.1 GerenciadorFase.cs

É o script que concentra a lógica do desafio (Figuras 1 a 4). Ele mantém referências ao jogador, ao painel, ao Block Man, às luzes, às janelas e aos elementos da interface, além das variáveis de estado: tempo restante, pistas encontradas, código digitado, pontuação e indicadores de vitória, derrota e painel aberto. No procedimento

Awake, define o tempo inicial em 120 segundos e reduz a intensidade das luzes a 30% do valor original, simulando o apagão. No Start, oculta as telas que só aparecem ao final, prepara os textos das quatro pistas e faz o Block Man dar as boas-vindas.

A cada quadro, o procedimento Update subtrai o tempo do quadro (Time.deltaTime) do tempo restante, verifica se ele chegou a zero, chama DarDicas, trata a digitação quando o painel está aberto, acende as luzes após a vitória, trata a tecla R nas telas finais e atualiza a interface (UNITY TECHNOLOGIES, [s. d.]). O procedimento LerPista registra cada bilhete uma única vez. AbrirPainel, FecharPainel, ApertarTecla, DigitarNoPainel, AdicionarDigito, AtualizarVisor e Confirmar organizam a interação com o teclado numérico. Vencer e Perder encerram a fase, Reiniciar recarrega a cena e IrParaProximaFase carrega a cena seguinte.

3.2 IntroducaoHistoria.cs

Conduz a abertura narrativa (Figuras 5 a 7). O vetor falas armazena as seis mensagens do Block Man, exibidas letra por letra a 35 caracteres por segundo. O jogador pode pressionar Espaço ou Enter para completar a fala em exibição ou avançar para a próxima. Se não o fizer, a fala seguinte surge automaticamente após uma pausa de 2,5 segundos, e a tecla Esc encerra a introdução.

Enquanto o texto avança, a câmera se aproxima gradualmente de um ponto definido para cada fala e o Block Man movimenta cabeça, boca e braço. O procedimento Efeitos produz relâmpagos em intervalos aleatórios entre 3 e 6 segundos, luzes piscando na segunda fala e faíscas no painel seguidas de apagão a partir da terceira. Ao final das falas, Terminar carrega a cena da Fase 1.

3.3 Camera.cs

Lê os movimentos do mouse nos eixos Mouse X e Mouse Y, multiplica-os pelas sensibilidades e acumula os valores em rotationX e rotationY (Figuras 8 e 9). O giro horizontal é aplicado ao corpo do personagem e a inclinação vertical, à câmera. O valor vertical é restringido entre –90° e 90° pelo método Mathf.Clamp, o que impede a visão de completar uma volta. No procedimento LateUpdate, a câmera acompanha a posição da cabeça do personagem e, no Start, o cursor é travado na janela por meio de Cursor.lockState.

3.4 Pegar

Implementado na classe Pegar, que aparece no arquivo PickupSystem.cs das capturas (Figuras 10 a 12). A cada quadro, o script verifica se a tecla E foi pressionada. Se não houver objeto guardado, o procedimento Pegar_ lança a partir da câmera um raio de alcance 3 (Physics.Raycast). Se o alvo possuir Rigidbody, o objeto é guardado, torna-se cinemático, tem o Collider desativado e passa a ser filho da mão do personagem.

Caso já exista um objeto guardado, o procedimento Soltar desfaz esses passos: remove o vínculo com a mão, reativa o Collider, devolve o comportamento físico e libera a referência. Mensagens Debug.Log registram cada etapa e facilitam a depuração.

3.5 PlayerMovement.cs

Obtém o CharacterController no Start. A cada quadro, lê os eixos Horizontal e Vertical, combina as direções direita e frente do personagem e aplica um deslocamento proporcional à velocidade (5 unidades por segundo) e ao tempo do quadro (Figuras 13 e 14). Em seguida, trata a gravidade: se o personagem está no chão e a velocidade vertical é negativa, ela é fixada em –2 para mantê-lo aderido ao solo. Depois, acumula a aceleração da gravidade (–9,81) e aplica o deslocamento vertical.

4 ESTRUTURAS DE PROGRAMAÇÃO UTILIZADAS

O Quadro 2 relaciona as estruturas de programação trabalhadas na disciplina às suas aplicações nos scripts do jogo.

Quadro 2 – Estruturas de programação identificadas nos scripts

Estrutura	Aplicação nos scripts
Variáveis	Armazenam tempo restante, pontuação, pistas encontradas, código digitado, estado da fase (venceu, perdeu, painelAberto), rotações da câmera, velocidade e gravidade.
Vetores	Guardam as pistas (string[4]), os textos de pista, as luzes, as janelas, as falas, os pontos de câmera e as intensidades originais das luzes.

Estrutura	Aplicação nos scripts
Operadores aritméticos	Subtraem o tempo do quadro, multiplicam o tempo restante por 10 para obter os pontos, calculam minutos e segundos do relógio (divisão e resto por 60) e o deslocamento (direção × velocidade × tempo).
Operadores relacionais e lógicos	Comparam tempo, códigos e estados, por exemplo tempoRestante <= 90 && deuDica1 == false.
Condicionais if/else	Decidem vitória ou derrota, tecla C, OK ou dígito, pista nova ou repetida, objeto atingido pelo raio e existência de objeto guardado.
Repetição for	Percorre as luzes (guardar e restaurar a intensidade), os textos das pistas, as janelas e as posições do visor do painel.
Repetição foreach	Percorre os caracteres digitados no teclado do computador em DigitarNoPainel.
Procedimentos	Métodos separados, com ou sem parâmetros, executam tarefas específicas (Quadro 3).

Fonte: Elaborado pelos autores (2026).

4.1 Procedimentos e parâmetros

Em C#, métodos declarados com void não retornam valor e correspondem aos procedimentos do pseudocódigo. Os métodos desenvolvidos para o jogo nos trechos analisados seguem esse modelo, com ou sem parâmetros, e cada um assume uma tarefa específica, conforme o Quadro 3. Procedimentos com parâmetros, como LerPista, ApertarTecla, AdicionarDigito, MostrarAviso e AnimarBlockMan, reaproveitam a mesma lógica para entradas diferentes.

Quadro 3 – Procedimentos dos scripts do jogo

Script	Procedimento	Parâmetro	Finalidade
GerenciadorFase	Awake e Start	—	Definem o tempo inicial, escurecem as luzes, ocultam telas finais e iniciam a fala de abertura.
GerenciadorFase	Update	—	Atualiza o tempo e chama os demais procedimentos a cada quadro.
GerenciadorFase	DarDicas	—	Emite dicas quando restam 90 s e 45 s.

Script	Procedimento	Parâmetro	Finalidade
GerenciadorFase	AtualizarInterface	—	Atualiza o relógio (mm:ss), o HUD e o texto de interação.
GerenciadorFase	MostrarAviso	string texto	Exibe um aviso temporário por 3 s.
GerenciadorFase	LerPista	Pista pista	Registra o bilhete encontrado, uma única vez.
GerenciadorFase	AbrirPainel e FecharPainel	—	Abrem e fecham o teclado numérico e travam ou liberam o movimento.
GerenciadorFase	ApertarTecla	string tecla	Trata as teclas C, OK e os dígitos do painel.
GerenciadorFase	DigitarNoPainel	—	Lê o teclado do computador (dígitos, apagar, Enter e Esc).
GerenciadorFase	AdicionarDigito	string digito	Acrescenta um dígito se ainda houver espaço.
GerenciadorFase	AtualizarVisor	—	Monta o visor, com “_” nas posições vazias.
GerenciadorFase	Confirmar	—	Compara o código digitado com o código correto.
GerenciadorFase	Vencer e Perder	—	Encerram a fase em vitória ou derrota.
GerenciadorFase	AcenderLuzes	—	Aumenta o brilho das luzes após a vitória.
GerenciadorFase	Reiniciar e IrParaProximaFase	—	Recarregam a cena atual ou carregam a próxima.
IntroducaoHistoria	ProximaFala e Terminar	—	Avançam as falas e carregam a cena da Fase 1.
IntroducaoHistoria	MoverCamera	—	Aproxima a câmera do ponto da fala atual.
IntroducaoHistoria	AnimarBlockMan	bool falando	Anima cabeça, boca e braço do personagem.
IntroducaoHistoria	Efeitos e AtualizarInterface	—	Produzem relâmpagos, piscadas, apagão e o texto digitado.
Camera	Start, Update e LateUpdate	—	Travam o cursor, giram a visão e seguem a cabeça do personagem.

Script	Procedimento	Parâmetro	Finalidade
Pegar	Update, Pegar_ e Soltar	—	Detectam a tecla E, pegam e soltam o objeto.
PlayerMovement	Start e Update	—	Obtêm o controlador e movem o personagem com gravidade.

Fonte: Elaborado pelos autores (2026).

5 PSEUDOCÓDIGOS DOS PRINCIPAIS ALGORITMOS

Os quadros a seguir apresentam, em pseudocódigo, os algoritmos principais do jogo. O procedimento Update da Unity é executado repetidamente, uma vez por quadro, e por isso é descrito como repetição a cada quadro.

5.1 Controle do tempo e dicas

O Quadro 4 descreve o cronômetro regressivo e o encerramento da fase por falta de tempo. O Quadro 5 descreve as dicas dadas pelo Block Man conforme o tempo passa.

Quadro 4 – Pseudocódigo do controle de tempo e do encerramento da fase

```
A CADA QUADRO (procedimento Update)
    SE a fase não foi vencida E não foi perdida ENTÃO
        tempo restante <- tempo restante - tempo do quadro
        SE tempo restante <= 0 ENTÃO
            tempo restante <- 0
            executar Perder
        FIM SE
        executar DarDicas
    FIM SE
    SE o painel está aberto ENTÃO
        executar DigitarNoPainel
    FIM SE
    SE a fase foi vencida ENTÃO
        executar AcenderLuzes
    FIM SE
    SE (a fase foi vencida OU perdida) E o jogador pressionou R ENTÃO
        executar Reiniciar
    FIM SE
    executar AtualizarInterface
```

Fonte: Elaborado pelos autores (2026).

Quadro 5 – Pseudocódigo das dicas do Block Man

```
PROCEDIMENTO DarDicas
    SE tempo restante <= 90 E dica 1 não foi dada ENTÃO
        marcar dica 1 como dada
        SE pistas encontradas < 2 ENTÃO
            Block Man fala a dica sobre onde procurar os bilhetes
        FIM SE
    FIM SE
    SE tempo restante <= 45 E dica 2 não foi dada ENTÃO
        marcar dica 2 como dada
        Block Man fala a dica sobre a ordem das pistas
    FIM SE
FIM PROCEDIMENTO
```

Fonte: Elaborado pelos autores (2026).

5.2 Leitura e registro de pistas

Cada bilhete só é contabilizado uma vez: ao lê-lo novamente, o jogo apenas exibe o texto da pista outra vez (Quadro 6).

Quadro 6 – Pseudocódigo da leitura e do registro de pistas

```
PROCEDIMENTO LerPista(pista)
    posição <- número da pista - 1
    SE a posição do vetor de pistas está vazia ENTÃO
        guardar o texto da pista nessa posição
        pistas encontradas <- pistas encontradas + 1
        atualizar o texto da interface e o contador "Pistas n/4"
        SE pistas encontradas = 4 ENTÃO
            Block Man informa que o código já pode ser digitado
        SENÃO SE pistas encontradas = 1 ENTÃO
            Block Man incentiva a continuar a busca
        FIM SE
    SENÃO
        mostrar novamente o texto da pista já encontrada
    FIM SE
FIM PROCEDIMENTO
```

Fonte: Elaborado pelos autores (2026).

5.3 Entrada e confirmação do código

O Quadro 7 mostra o tratamento das teclas do painel e o Quadro 8, a montagem do visor, que usa uma repetição para exibir os dígitos já digitados e traços nas posições vazias.

Quadro 7 – Pseudocódigo da entrada e da confirmação do código

```
PROCEDIMENTO ApertarTecla(tecla)
    SE tecla = "C" ENTÃO
        digitado <- ""
    SENÃO SE tecla = "OK" ENTÃO
        executar Confirmar
    SENÃO
        executar AdicionarDigito(tecla)
    FIM SE
    executar AtualizarVisor
FIM PROCEDIMENTO

PROCEDIMENTO AdicionarDigito(digito)
    SE tamanho de digitado < tamanho do código correto ENTÃO
        digitado <- digitado + digito
    FIM SE
FIM PROCEDIMENTO

PROCEDIMENTO Confirmar
    SE digitado = código correto ENTÃO
        executar Vencer
    SENÃO
        digitado <- ""
        mostrar "Código incorreto!" e Block Man orienta reler as pistas
    FIM SE
FIM PROCEDIMENTO
```

Fonte: Elaborado pelos autores (2026).

Quadro 8 – Pseudocódigo da atualização do visor do painel

```
PROCEDIMENTO AtualizarVisor
    visor <- ""
    PARA i DE 0 ATÉ tamanho do código correto - 1 FAÇA
        SE i < tamanho de digitado ENTÃO
            visor <- visor + digitado[i] + " "
        SENÃO
            visor <- visor + "_ "
        FIM SE
    FIM PARA
    exibir visor na tela do painel
FIM PROCEDIMENTO
```

Fonte: Elaborado pelos autores (2026).

5.4 Vitória e pontuação

Ao acertar o código, a rotina de vitória calcula a pontuação e atualiza o melhor tempo, que fica salvo no computador do jogador por meio de PlayerPrefs (Quadro 9).

Quadro 9 – Pseudocódigo da rotina de vitória e do cálculo da pontuação

```
PROCEDIMENTO Vencer
    marcar a fase como vencida e fechar o painel
    pontos <- arredondar(tempo restante * 10)
    tempo gasto <- tempo total - tempo restante
    melhor tempo <- ler melhor tempo salvo (padrão 9999)
    SE tempo gasto < melhor tempo ENTÃO
        melhor tempo <- tempo gasto
        salvar o melhor tempo no computador
    FIM SE
    bloquear o movimento do jogador
    mudar a aparência da fábrica ("amanhecer")
    exibir tela de vitória com tempo, melhor tempo e pontuação
FIM PROCEDIMENTO
```

Fonte: Elaborado pelos autores (2026).

5.5 Pegar ou soltar um objeto

Quadro 10 – Pseudocódigo de pegar e soltar um objeto

```
A CADA QUADRO (procedimento Update)
    SE o jogador pressionou E ENTÃO
        SE não há objeto guardado ENTÃO
            executar Pegar_
        SENÃO
            executar Soltar
        FIM SE
    FIM SE

PROCEDIMENTO Pegar_
    lançar um raio da câmera, para frente, com alcance de 3
    SE o raio atingiu algo ENTÃO
        SE o alvo possui Rigidbody ENTÃO
            guardar o objeto
            tornar o objeto cinemático e desativar seu Collider
            prender o objeto à mão, na posição local (0, 0, 0)
        FIM SE
    FIM SE
FIM PROCEDIMENTO

PROCEDIMENTO Soltar
    desvincular o objeto da mão
    reativar o Collider e a física do objeto
    objeto guardado <- nenhum
FIM PROCEDIMENTO
```

Fonte: Elaborado pelos autores (2026).

5.6 Movimentação do personagem

Quadro 11 – Pseudocódigo da movimentação do personagem

```
A CADA QUADRO (procedimento Update)
    x <- valor do eixo Horizontal
    z <- valor do eixo Vertical
    direção <- direita do personagem * x + frente do personagem * z
    mover(direção * velocidade * tempo do quadro)
    SE está no chão E velocidade vertical < 0 ENTÃO
        velocidade vertical <- -2
    FIM SE
    velocidade vertical <- velocidade vertical + gravidade * tempo do quadro
    mover(velocidade * tempo do quadro)
```

Fonte: Elaborado pelos autores (2026).

5.7 Rotação da câmera

Quadro 12 – Pseudocódigo da rotação da câmera em primeira pessoa

```
A CADA QUADRO (procedimento Update)
    deltaV <- valor do eixo Mouse Y * sensibilidade Y
    deltaH <- valor do eixo Mouse X * sensibilidade X
    rotação X <- rotação X + deltaH
    rotação Y <- rotação Y + deltaV
    rotação Y <- limitar rotação Y entre -90 e 90
    girar o corpo do personagem em torno do eixo vertical (rotação X)
    girar a câmera: inclinação = -rotação Y e giro = rotação X

DEPOIS DO QUADRO (procedimento LateUpdate)
    posição da câmera <- posição da cabeça do personagem
```

Fonte: Elaborado pelos autores (2026).

5.8 Apresentação das falas

A comparação entre o número de letras já liberadas e o tamanho da fala define se o texto ainda está sendo digitado (Quadro 13).

Quadro 13 – Pseudocódigo da apresentação das falas letra por letra

```
A CADA QUADRO (procedimento Update)
    SE o jogador pressionou ESC ENTÃO
        executar Terminar
    FIM SE
    tempo da linha <- tempo da linha + tempo do quadro
    fala <- falas[índice]
    letras <- parte inteira de (tempo da linha * velocidade do texto)
    digitando <- letras < tamanho da fala
    SE não está digitando ENTÃO
        tempo de pausa <- tempo de pausa + tempo do quadro
    FIM SE
    SE o jogador pressionou Espaço ou Enter ENTÃO
        SE está digitando ENTÃO
            mostrar a fala completa
        SENÃO
            executar ProximaFala
        FIM SE
    FIM SE
    SE não está digitando E tempo de pausa > 2,5 ENTÃO
        executar ProximaFala
    FIM SE
    exibir os primeiros "letras" caracteres da fala

PROCEDIMENTO ProximaFala
    índice <- índice + 1
    SE índice >= quantidade de falas ENTÃO
        executar Terminar (carrega a cena da Fase 1)
    SENÃO
        zerar tempo da linha e tempo de pausa
    FIM SE
FIM PROCEDIMENTO
```

Fonte: Elaborado pelos autores (2026).

6 RESULTADOS

6.1 Cenários de verificação

O Quadro 14 descreve o resultado que a lógica dos scripts produz em cada situação de jogo e indica o procedimento responsável, o que permite relacionar cada comportamento ao trecho de código correspondente.

Quadro 14 – Cenários de verificação e resultados esperados

Nº	Situação	Procedimento	Resultado esperado
1	Iniciar a fase.	Awake e Start	O tempo começa em 120 s, as telas finais ficam ocultas e a interface mostra “Pistas 0/4”.
2	Encontrar um bilhete.	LerPista	A pista é registrada uma única vez, o contador aumenta e o texto da interface é atualizado.
3	Encontrar os quatro bilhetes.	LerPista	O Block Man avisa que todas as pistas foram achadas e o código pode ser digitado.
4	Digitar mais dígitos que o tamanho do código.	AdicionarDigito	Os dígitos excedentes são ignorados.
5	Digitar um código incorreto.	Confirmar	A entrada é limpa, aparece “Código incorreto!” e o jogador pode tentar de novo enquanto houver tempo.
6	Digitar o código correto.	Confirmar e Vencer	A fase é marcada como vencida, a pontuação é calculada e a tela de vitória aparece.
7	Deixar o tempo chegar a zero.	Update e Perder	O tempo é fixado em 0 e a tela de derrota aparece.
8	Pressionar E diante de um objeto com Rigidbody.	Pegar_	O objeto é preso à mão e seu Collider é desativado.
9	Pressionar E com um objeto guardado.	Soltar	O objeto é desvinculado da mão e sua colisão e física são reativadas.
10	Mover o mouse e usar as teclas de direção.	Update (Camera e PlayerMovement)	A visão gira com limite vertical de –90° a 90° e o personagem se desloca na direção para a qual olha.

Fonte: Elaborado pelos autores (2026).

6.2 Cálculo da pontuação

A pontuação depende apenas do tempo que sobra: pontos = arredondamento de (tempo restante × 10), e o tempo gasto é a diferença entre o tempo total (120 s) e o tempo restante. O Quadro 15 apresenta valores obtidos aplicando essas fórmulas do código a quatro situações de término da fase. Quanto mais rápido o jogador digita o código correto, maior a pontuação.

Quadro 15 – Pontuação obtida conforme o tempo restante na vitória

Tempo restante (s)	Tempo gasto (s)	Pontuação
90	30	900
60	60	600
45	75	450
10	110	100

Fonte: Elaborado pelos autores (2026).

7 CONCLUSÃO

A implementação analisada demonstra a aplicação prática de algoritmos no jogo Protocolo Arcor. A lógica foi dividida em scripts e procedimentos com responsabilidades específicas: GerenciadorFase coordena o desafio e suas condições de encerramento, IntroducaoHistoria apresenta a narrativa, Camera e PlayerMovement controlam a experiência em primeira pessoa e Pegar implementa a interação com objetos.

Os scripts utilizam variáveis e vetores para armazenar estados e dados, operadores para cálculos e comparações, estruturas condicionais para decidir o rumo da partida e estruturas de repetição para percorrer coleções e entradas. A separação em procedimentos, com e sem parâmetros, torna o código modular e facilita localizar cada parte da lógica, e as decisões escritas com if/else explícitos, sem operador ternário, atendem à orientação da disciplina.

Dessa forma, o trabalho relaciona os conteúdos de Algoritmos e Lógica de Programação a funcionalidades concretas do jogo, com pseudocódigos que descrevem cada algoritmo e resultados que mostram seu efeito na experiência do jogador.

REFERÊNCIAS

FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO. **Arcor Play – Jornada pelas Marcas e Produtos Arcor**: projeto interdisciplinar – 1º CCOMP, 2º semestre 2026. Versão final. São Paulo: FECAP, 2026.

UNITY TECHNOLOGIES. **Unity Scripting API**. [S. l.]: Unity Technologies, [s. d.]. Disponível em: https://docs.unity3d.com/ScriptReference/. Acesso em: 6 nov. 2026.

APÊNDICE A – EVIDÊNCIAS DOS SCRIPTS (CAPTURAS DE TELA)

As figuras a seguir reproduzem as capturas de tela do código-fonte dos scripts analisados neste relatório, no ambiente de desenvolvimento.

Figura 1 – Código-fonte de GerenciadorFase.cs (linhas 1 a 100)

GerenciadorFase.cs (parte 1/4)

```
using TMPro;
using UnityEngine;
using UnityEngine.SceneManagement;


public class GerenciadorFase : MonoBehaviour
{
    // ---------- referências do mundo (arrastadas na cena) ----------
    public Jogador jogador;
    public Painel painel;
    public BlockMan blockMan;
    public Renderer telaPainel;
    public Light[] luzes;
    public Renderer[] janelas;

    // ---------- referências da interface (Canvas) ----------
    public GameObject hud;              // relógio, pistas e mira
    public TMP_Text textoRelogio;
    public TMP_Text textoTituloPistas;
    public TMP_Text[] textosPistas;
    public TMP_Text textoInteracao;
    public GameObject caixaInteracao;
    public TMP_Text textoAviso;
    public GameObject caixaAviso;

    public GameObject telaPainelCodigo;  // teclado numérico
    public TMP_Text textoVisor;
    public TMP_Text textoMensagemPainel;

    public GameObject telaVitoria;
    public TMP_Text textoResultado;
    public GameObject botaoProximaFase;

    public GameObject telaDerrota;

    // ---------- configurações ----------
    public float tempoTotal = 120f;
    public string proximaCena = "Fase2_ZonaPlutonica";

    // ---------- estado do jogo ----------
    float tempoRestante;
    string[] pistas = new string[4];
    int pistasEncontradas = 0;

    bool painelAberto = false;
    bool venceu = false;
    bool perdeu = false;
    int pontos = 0;
    float tempoGasto = 0f;
    float melhorTempo = 0f;

    string digitado = "";
    float tempoMsgPainel = 0f;
    float tempoAviso = 0f;

    bool deuDica1 = false;
    bool deuDica2 = false;

    // as luzes começam fracas e acendem quando vence
    float brilho = 0.3f;
    float[] intensidadeOriginal;

    void Awake()
    {
        tempoRestante = tempoTotal;

        // guarda a intensidade das luzes e deixa tudo fraco (apagão)
        intensidadeOriginal = new float[luzes.Length];
        for (int i = 0; i < luzes.Length; i++)
        {
            intensidadeOriginal[i] = luzes[i].intensity;
            luzes[i].intensity = intensidadeOriginal[i] * brilho;
        }
    }

    void Start()
    {
        // esconde as telas que só aparecem depois
        telaPainelCodigo.SetActive(false);
        telaVitoria.SetActive(false);
        telaDerrota.SetActive(false);
        caixaAviso.SetActive(false);
        textoMensagemPainel.text = "";

        textoTituloPistas.text = "Pistas 0/4";
        for (int i = 0; i < textosPistas.Length; i++)
        {
            textosPistas[i].text = (i + 1) + ". ???";
        }

        blockMan.Falar("Olá, técnico Fúlvio! Um apagão travou a fábrica. Ache os bilhetes com as pistas e digite o código no painel antes que o tempo acabe!", 9f);
    }

    void Update()
    {
        if (venceu == false && perdeu == false)
        {
            tempoRestante = tempoRestante - Time.deltaTime;

            if (tempoRestante <= 0)
```

Fonte: Elaborado pelos autores (2026).

Figura 2 – Código-fonte de GerenciadorFase.cs (linhas 101 a 200)



GerenciadorFase.cs (parte 2/4)

```
            {
                tempoRestante = 0;
                Perder();
            }

            DarDicas();
        }

        if (painelAberto == true)
        {
            DigitarNoPainel();
        }

        if (venceu == true)
        {
            AcenderLuzes();
        }

        if ((venceu == true || perdeu == true) && Input.GetKeyDown(KeyCode.R))
        {
            Reiniciar();
        }

        AtualizarInterface();
    }

    // dicas do Block Man conforme o tempo passa
    void DarDicas()
    {
        if (tempoRestante <= 90 && deuDica1 == false)
        {
            deuDica1 = true;
            if (pistasEncontradas < 2)
            {
                blockMan.Falar("Dica: procure bilhetes em cima das mesas, da caixa e da esteira. Cada um
revela um dígito do código.", 7f);
            }
        }
        if (tempoRestante <= 45 && deuDica2 == false)
        {
            deuDica2 = true;
            blockMan.Falar("Faltam 45 segundos! Cada dígito depende do anterior, use as pistas na ordem.",
7f);
        }
    }

    // ---------- interface ----------

    // atualiza os textos que mudam o tempo todo
    void AtualizarInterface()
    {
        // relógio no formato 01:45
        int segundos = Mathf.CeilToInt(tempoRestante);
        textoRelogio.text = (segundos / 60).ToString("00") + ":" + (segundos % 60).ToString("00");

        // o HUD some quando o jogo termina
        hud.SetActive(venceu == false && perdeu == false);

        // texto de interação ("[E] Ler bilhete"); some quando o painel está aberto
        bool mostrarInteracao = painelAberto == false && jogador.textoInteracao != "";
        caixaInteracao.SetActive(mostrarInteracao);
        if (mostrarInteracao == true)
        {
            textoInteracao.text = jogador.textoInteracao;
        }

        // aviso rápido no fim da tela
        caixaAviso.SetActive(Time.time < tempoAviso);

        // mensagem de erro do painel some depois de um tempo
        if (Time.time >= tempoMsgPainel)
        {
            textoMensagemPainel.text = "";
        }
    }

    void MostrarAviso(string texto)
    {
        textoAviso.text = texto;
        tempoAviso = Time.time + 3f;
    }

    // ---------- pistas ----------

    public void LerPista(Pista pista)
    {
        int posicao = pista.numero - 1;

        if (pistas[posicao] == null)
        {
            pistas[posicao] = pista.texto;
            pistasEncontradas = pistasEncontradas + 1;

            textosPistas[posicao].text = pista.numero + ". " + pista.texto;
            textoTituloPistas.text = "Pistas " + pistasEncontradas + "/4";
            MostrarAviso("Pista " + pista.numero + " encontrada!");

            if (pistasEncontradas == 4)
            {
                blockMan.Falar("Você achou todas as pistas! Corra até o painel de controle e digite o
código.", 7f);
            }
            else if (pistasEncontradas == 1)
```

Fonte: Elaborado pelos autores (2026).

Figura 3 – Código-fonte de GerenciadorFase.cs (linhas 201 a 300)

GerenciadorFase.cs (parte 3/4)

```
            {
                blockMan.Falar("Boa! Continue procurando os outros bilhetes pela sala.", 5f);
            }
        }
        else
        {
            MostrarAviso("Pista " + pista.numero + ": " + pista.texto);
        }
    }

    // ---------- painel de código ----------

    public void AbrirPainel()
    {
        painelAberto = true;
        digitado = "";
        AtualizarVisor();
        telaPainelCodigo.SetActive(true);
        jogador.podeMover = false;
        jogador.TravarMouse(false);
    }

    // ligado ao botão "Fechar"
    public void FecharPainel()
    {
        painelAberto = false;
        telaPainelCodigo.SetActive(false);
        jogador.podeMover = true;
        jogador.TravarMouse(true);
    }

    // ligado aos botões do teclado numérico (cada botão manda o seu texto)
    public void ApertarTecla(string tecla)
    {
        if (tecla == "C")
        {
            digitado = "";
        }
        else if (tecla == "OK")
        {
            Confirmar();
        }
        else
        {
            AdicionarDigito(tecla);
        }
        AtualizarVisor();
    }

    // também dá para digitar no teclado do computador
    void DigitarNoPainel()
    {
        foreach (char c in Input.inputString)
        {
            if (c >= '0' && c <= '9')
            {
                AdicionarDigito(c.ToString());
            }
            else if (c == '\b' && digitado.Length > 0)
            {
                digitado = digitado.Substring(0, digitado.Length - 1);
            }
            else if (c == '\n' || c == '\r')
            {
                Confirmar();
            }
        }
        AtualizarVisor();

        if (Input.GetKeyDown(KeyCode.Escape))
        {
            FecharPainel();
        }
    }

    void AdicionarDigito(string digito)
    {
        if (digitado.Length < painel.codigoCorreto.Length)
        {
            digitado = digitado + digito;
        }
    }

    // mostra o que foi digitado, com "_" nas posições que faltam
    void AtualizarVisor()
    {
        string visor = "";
        for (int i = 0; i < painel.codigoCorreto.Length; i++)
        {
            if (i < digitado.Length)
            {
                visor = visor + digitado[i] + " ";
            }
            else
            {
                visor = visor + "_ ";
            }
        }
        textoVisor.text = visor;
    }
```

Fonte: Elaborado pelos autores (2026).

Figura 4 – Código-fonte de GerenciadorFase.cs (linhas 301 a 398)

GerenciadorFase.cs (parte 4/4)

```
301     void Confirmar()
302     {
303         if (digitado == painel.codigoCorreto)
304         {
305             Vencer();
306         }
307         else
308         {
309             digitado = "";
310             textoMensagemPainel.text = "Código incorreto!";
311             tempoMsgPainel = Time.time + 1.5f;
312             blockMan.Falar("Código incorreto. Releia as pistas com calma!", 5f);
313         }
314     }
315
316     // ---------- fim de fase ----------
317
318     void Vencer()
319     {
320         venceu = true;
321         painelAberto = false;
322         painel.resolvido = true;
323         pontos = Mathf.RoundToInt(tempoRestante * 10);
324
325         // guarda o melhor tempo
326         tempoGasto = tempoTotal - tempoRestante;
327         melhorTempo = PlayerPrefs.GetFloat("MelhorTempoFase1", 9999f);
328         if (tempoGasto < melhorTempo)
329         {
330             melhorTempo = tempoGasto;
331             PlayerPrefs.SetFloat("MelhorTempoFase1", melhorTempo);
332             PlayerPrefs.Save();
333         }
334
335         jogador.podeMover = false;
336         jogador.TravarMouse(false);
337
338         // a fábrica "amanhece"
339         telaPainel.material.color = new Color(0.2f, 1f, 0.5f);
340         for (int i = 0; i < janelas.Length; i++)
341         {
342             janelas[i].material.color = new Color(0.75f, 0.88f, 1f);
343         }
344         RenderSettings.ambientLight = new Color(0.5f, 0.5f, 0.55f);
345
346         // tela de vitória
347         telaPainelCodigo.SetActive(false);
348         textoResultado.text = "Seu tempo: " + tempoGasto.ToString("0.0") + " s\n"
349             + "Melhor tempo: " + melhorTempo.ToString("0.0") + " s\n"
350             + "Pontuação: " + pontos;
351         botaoProximaFase.SetActive(proximaCena != "");
352         telaVitoria.SetActive(true);
353
354         blockMan.Falar("Fábrica reativada! Na próxima parada, procure a fase bônus Central Bon o Bon para
ganhar a Bala 7 Belo.", 10f);
355     }
356
357     void Perder()
358     {
359         perdeu = true;
360         painelAberto = false;
361         jogador.podeMover = false;
362         jogador.TravarMouse(false);
363
364         telaPainelCodigo.SetActive(false);
365         telaDerrota.SetActive(true);
366
367         blockMan.Falar("O tempo acabou... Tente de novo!", 6f);
368     }
369
370     void AcenderLuzes()
371     {
372         // o brilho sobe até 1 em uns 3 segundos
373         brilho = brilho + Time.deltaTime / 3f;
374         if (brilho > 1f)
375         {
376             brilho = 1f;
377         }
378
379         for (int i = 0; i < luzes.Length; i++)
380         {
381             luzes[i].intensity = intensidadeOriginal[i] * brilho;
382         }
383     }
384
385     // ---------- botões das telas de fim ----------
386
387     // ligado aos botões "Jogar de novo" e "Tentar de novo"
388     public void Reiniciar()
389     {
390         SceneManager.LoadScene(SceneManager.GetActiveScene().name);
391     }
392
393     // ligado ao botão "Próxima fase"
394     public void IrParaProximaFase()
395     {
396         SceneManager.LoadScene(proximaCena);
397     }
398 }
```

Fonte: Elaborado pelos autores (2026).

Figura 5 – Código-fonte de IntroducaoHistoria.cs (linhas 1 a 80)



IntroducaoHistoria.cs (parte 1/3)

```
1  using TMPro;
2  using UnityEngine;
3  using UnityEngine.SceneManagement;
4  using UnityEngine.UI;
5
6  public class IntroducaoHistoria : MonoBehaviour
7  {
8      public string proximaCena = "Fase1_Estacao7Belo";
9
10     public Camera cameraIntro;
11     public Transform[] pontos;      // um ponto de câmera para cada fala
12     public Light[] luzes;           // luzes da sala
13     public Renderer telaPainel;
14     public Light luzFaisca;         // luz amarela que pisca no painel
15
16     // partes do Block Man
17     public Transform cabeca;
18     public Transform boca;
19     public Transform bracoDir;
20
21
22     public TMP_Text textoFala;      // texto da fala, aparece letra por letra
23     public TMP_Text textoTitulo;    // "PROTOCOLO ARCOR" nos primeiros 4 segundos
24     public Image imagemFlash;       // clarão branco do relâmpago
25
26     public string[] falas = {
27         "Olá! Eu sou o Block Man, funcionário da fábrica da Arcor. Deixa eu te contar o que aconteceu hoje
à noite.",
28         "Uma tempestade atípica atingiu a região e sobrecarregou os nossos geradores...",
29         "Ouviu isso? Faíscas no painel principal! Tudo apagou e o Protocolo Arcor travou todas as estações
da fábrica.",
30         "Alguém precisa entrar às escuras e reativar cada setor: a Estação 7 Belo, a Zona Plutônica e a
Câmara da Tartaruga.",
31         "O técnico Fúlvio Marques se voluntariou. Você é o Fúlvio, e tem só 2 minutos em cada fase!",
32         "Eu vou te guiar pelo rádio. Boa sorte, Fúlvio!"
33     };
34
35     public float velocidadeTexto = 35f;   // letras por segundo
36
37     int indice = 0;
38     float tempoLinha = 0f;
39     float tempoPausa = 0f;
40     float tempoGeral = 0f;
41
42     float[] luzOriginal;
43     bool apagou = false;
44     float tempoPisca = 0f;
45     float relampago = 0f;
46     float proximoRelampago = 4f;
47
48     void Start()
49     {
50         luzOriginal = new float[luzes.Length];
51         for (int i = 0; i < luzes.Length; i++)
52         {
53             luzOriginal[i] = luzes[i].intensity;
54         }
55
56         telaPainel.material.color = new Color(0.2f, 1f, 0.5f);
57         luzFaisca.enabled = false;
58
59         cameraIntro.transform.position = pontos[0].position;
60         cameraIntro.transform.rotation = pontos[0].rotation;
61     }
62
63     void Update()
64     {
65         if (Input.GetKeyDown(KeyCode.Escape))
66         {
67             Terminar();
68             return;
69         }
70
71         tempoGeral = tempoGeral + Time.deltaTime;
72         tempoLinha = tempoLinha + Time.deltaTime;
73
74         string fala = falas[indice];
75         int letras = (int)(tempoLinha * velocidadeTexto);
76         bool digitando = letras < fala.Length;
77
78         if (digitando == false)
79         {
80             tempoPausa = tempoPausa + Time.deltaTime;
```

Fonte: Elaborado pelos autores (2026).

Figura 6 – Código-fonte de IntroducaoHistoria.cs (linhas 81 a 160)

IntroducaoHistoria.cs (parte 2/3)

```
81          }
82
83          // se ainda está digitando mostra tudo, senão vai para a próxima fala
84          if (Input.GetKeyDown(KeyCode.Space) || Input.GetKeyDown(KeyCode.Return))
85          {
86              if (digitando == true)
87              {
88                  tempoLinha = fala.Length / velocidadeTexto + 0.01f;
89              }
90              else
91              {
92                  ProximaFala();
93                  return;
94              }
95          }
96
97          // passa sozinho depois de uma pausa
98          if (digitando == false && tempoPausa > 2.5f)
99          {
100             ProximaFala();
101             return;
102         }
103
104         MoverCamera();
105         AnimarBlockMan(digitando);
106         Efeitos();
107         AtualizarInterface();
108     }
109
110     void ProximaFala()
111     {
112         indice = indice + 1;
113         if (indice >= falas.Length)
114         {
115             Terminar();
116             return;
117         }
118         tempoLinha = 0f;
119         tempoPausa = 0f;
120     }
121
122     void MoverCamera()
123     {
124         // a câmera vai chegando devagar no ponto da fala atual
125         Transform alvo = pontos[indice];
126         cameraIntro.transform.position = Vector3.Lerp(cameraIntro.transform.position, alvo.position,
Time.deltaTime * 0.8f);
127         cameraIntro.transform.rotation = Quaternion.Slerp(cameraIntro.transform.rotation, alvo.rotation,
Time.deltaTime * 1.2f);
128     }
129
130     void AnimarBlockMan(bool falando)
131     {
132         // a cabeça balança de leve
133         cabeca.localEulerAngles = new Vector3(Mathf.Sin(Time.time * 2f) * 3f, Mathf.Sin(Time.time * 1.3f)
* 8f, 0);
134
135         if (falando == true)
136         {
137             // a boca abre e fecha e o braço faz gestos
138             float abertura = 0.04f + Mathf.Abs(Mathf.Sin(Time.time * 18f)) * 0.08f;
139             boca.localScale = new Vector3(0.16f, abertura, 0.02f);
140             bracoDir.localEulerAngles = new Vector3(0, 0, -20f + Mathf.Sin(Time.time * 4f) * 20f);
141         }
142         else
143         {
144             boca.localScale = new Vector3(0.16f, 0.04f, 0.02f);
145             bracoDir.localEulerAngles = new Vector3(0, 0, 0);
146         }
147     }
148
149     void Efeitos()
150     {
151         // relâmpagos de vez em quando
152         relampago = relampago - Time.deltaTime * 2f;
153         if (relampago < 0)
154         {
155             relampago = 0;
156         }
157         proximoRelampago = proximoRelampago - Time.deltaTime;
158         if (proximoRelampago <= 0)
159         {
160             relampago = 0.5f;
```

Fonte: Elaborado pelos autores (2026).

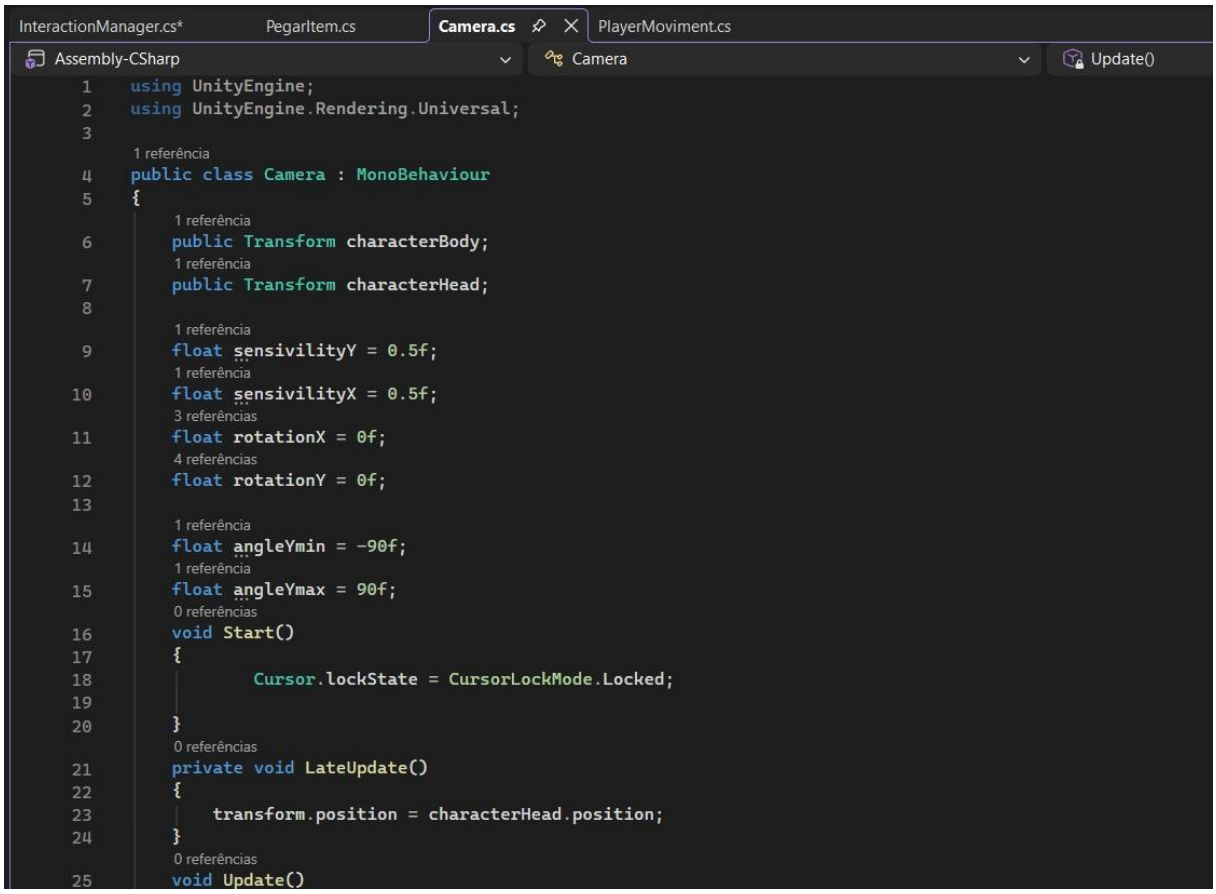
Figura 7 – Código-fonte de IntroducaoHistoria.cs (linhas 161 a 232)

IntroducaoHistoria.cs (parte 3/3)

```
161             proximoRelampago = Random.Range(3f, 6f);
162         }
163
164         // fala 2: as luzes ficam piscando
165         if (indice == 1)
166         {
167             tempoPisca = tempoPisca - Time.deltaTime;
168             if (tempoPisca <= 0)
169             {
170                 tempoPisca = 0.12f;
171                 for (int i = 0; i < luzes.Length; i++)
172                 {
173                     if (Random.value > 0.5f)
174                     {
175                         luzes[i].intensity = luzOriginal[i];
176                     }
177                     else
178                     {
179                         luzes[i].intensity = 0;
180                     }
181                 }
182             }
183         }
184
185         // fala 3: faíscas no painel e apagão
186         if (indice == 2 && tempoLinha < 3f)
187         {
188             luzFaisca.enabled = Random.value > 0.6f;
189         }
190         else
191         {
192             luzFaisca.enabled = false;
193         }
194
195         if (indice >= 2 && apagou == false)
196         {
197             apagou = true;
198             telaPainel.material.color = new Color(0.05f, 0.05f, 0.05f);
199             for (int i = 0; i < luzes.Length; i++)
200             {
201                 luzes[i].intensity = luzOriginal[i] * 0.3f;
202             }
203         }
204     }
205
206     void Terminar()
207     {
208         SceneManager.LoadScene(proximaCena);
209     }
210
211     // ---------- tela ----------
212
213     // atualiza os textos e o clarão a cada frame
214     void AtualizarInterface()
215     {
216         // clarão do relâmpago
217         imagemFlash.color = new Color(1, 1, 1, relampago * 0.6f);
218
219         // título nos primeiros 4 segundos, sumindo aos poucos
220         textoTitulo.gameObject.SetActive(tempoGeral < 4f);
221         textoTitulo.color = new Color(1, 1, 1, 1f - tempoGeral / 4f);
222
223         // texto aparecendo letra por letra
224         string fala = falas[indice];
225         int letras = (int)(tempoLinha * velocidadeTexto);
226         if (letras > fala.Length)
227         {
228             letras = fala.Length;
229         }
230         textoFala.text = fala.Substring(0, letras);
231     }
232 }
```

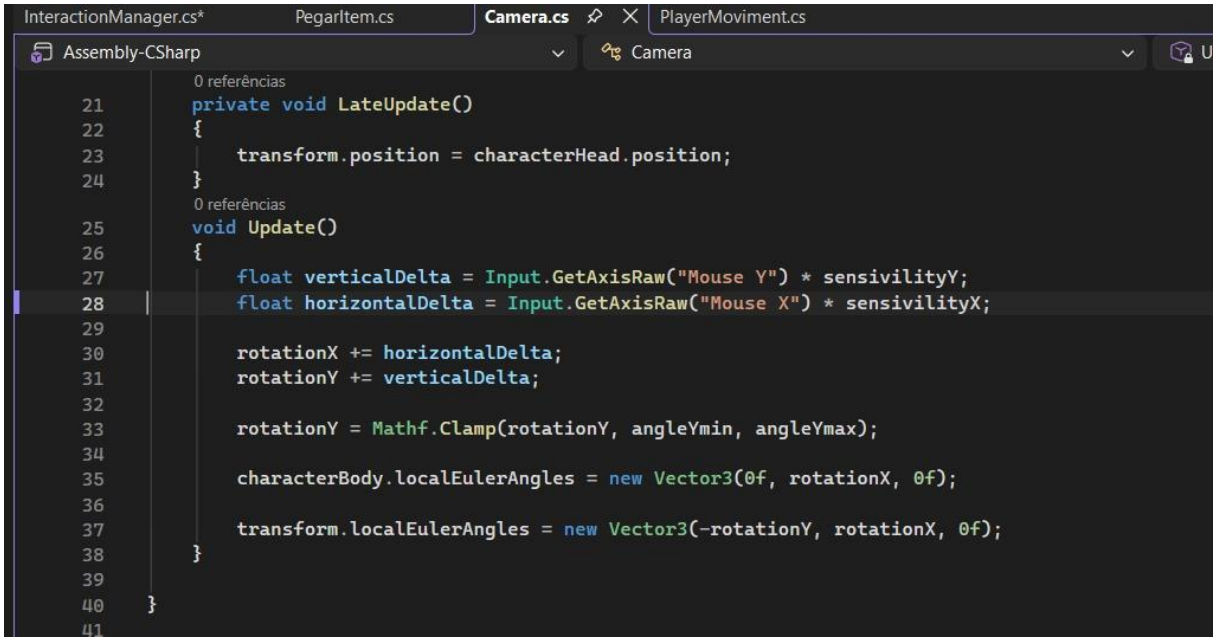
Fonte: Elaborado pelos autores (2026).

Figura 8 – Código-fonte de Camera.cs (linhas 1 a 25)



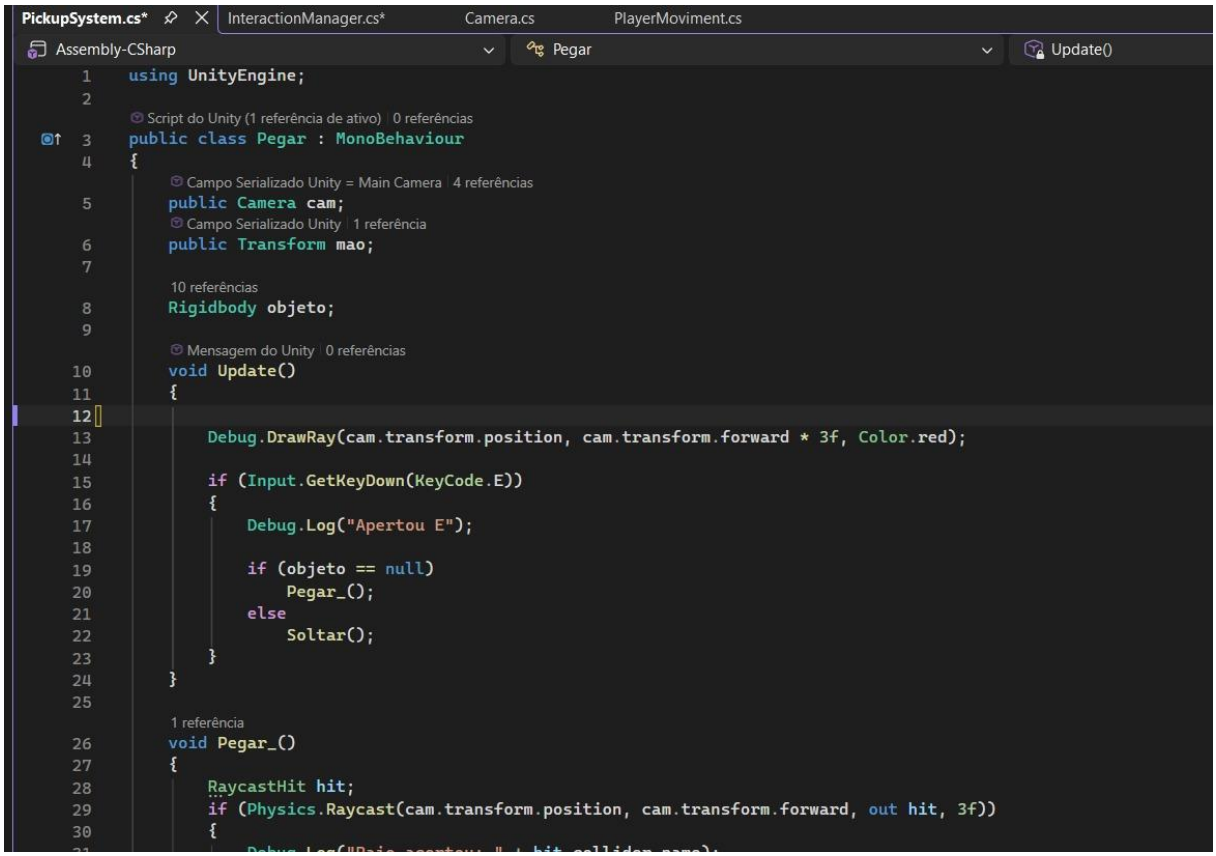
Fonte: Elaborado pelos autores (2026).

Figura 9 – Código-fonte de Camera.cs (linhas 21 a 41)



Fonte: Elaborado pelos autores (2026).

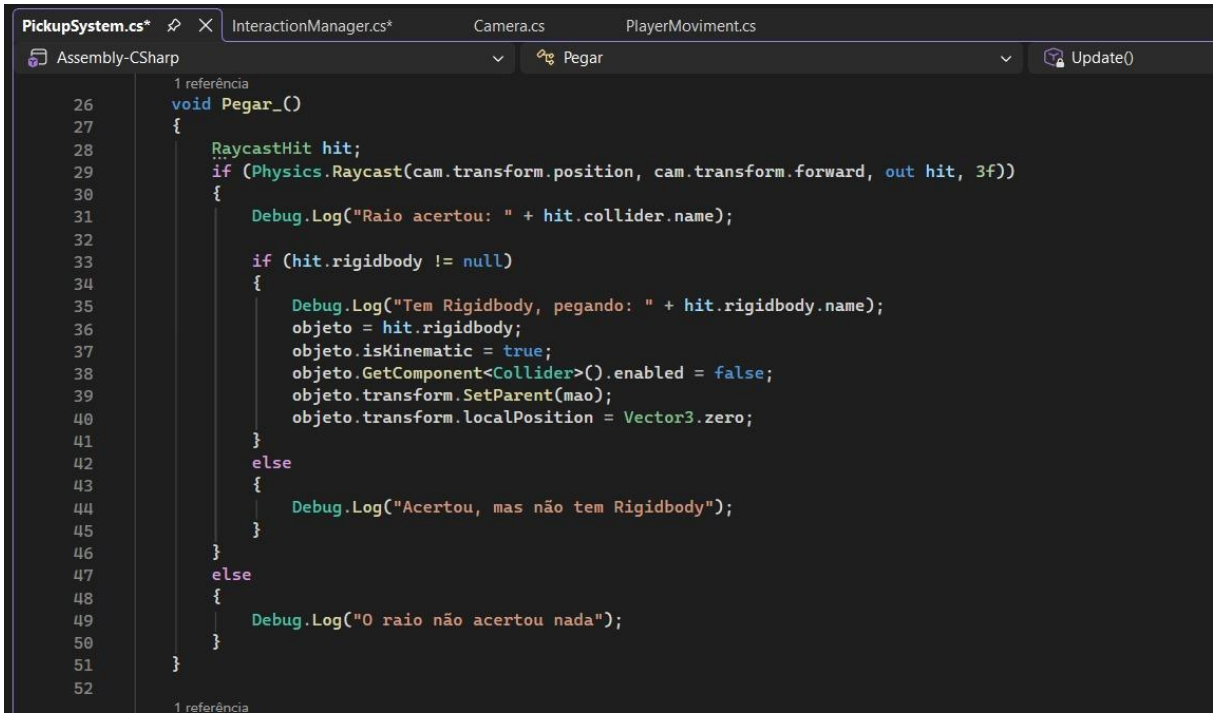
Figura 10 – Código-fonte da classe Pegar (linhas 1 a 31)



```
PickupSystem.cs*  InteractionManager.cs*  Camera.cs  PlayerMoviment.cs
Assembly-CSharp  Pegar  Update()
1  using UnityEngine;
2
   Script do Unity (1 referência de ativo) | 0 referências
3  public class Pegar : MonoBehaviour
4  {
       Campo Serializado Unity = Main Camera | 4 referências
5      public Camera cam;
       Campo Serializado Unity | 1 referência
6      public Transform mao;
7
       10 referências
8      Rigidbody objeto;
9
       Mensagem do Unity | 0 referências
10     void Update()
11     {
12
13         Debug.DrawRay(cam.transform.position, cam.transform.forward * 3f, Color.red);
14
15         if (Input.GetKeyDown(KeyCode.E))
16         {
17             Debug.Log("Apertou E");
18
19             if (objeto == null)
20                 Pegar_();
21             else
22                 Soltar();
23         }
24     }
25
       1 referência
26     void Pegar_()
27     {
28         RaycastHit hit;
29         if (Physics.Raycast(cam.transform.position, cam.transform.forward, out hit, 3f))
30         {
```

Fonte: Elaborado pelos autores (2026).

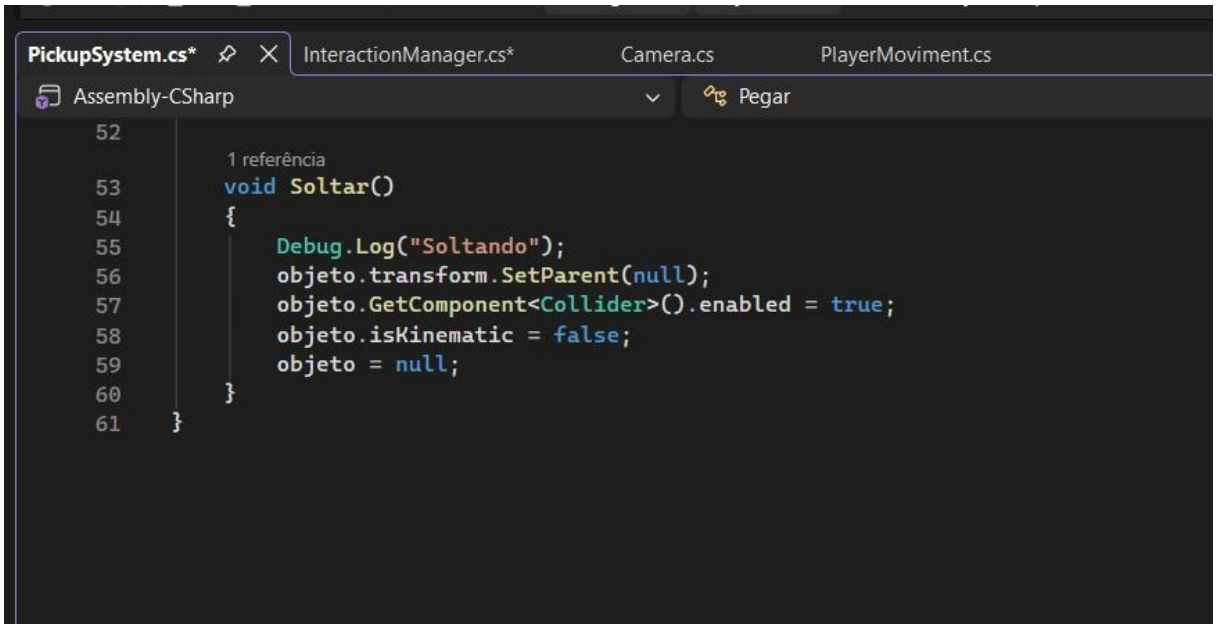
Figura 11 – Código-fonte da classe Pegar: método Pegar_ (linhas 26 a 52)



```
PickupSystem.cs*  InteractionManager.cs*  Camera.cs  PlayerMoviment.cs
Assembly-CSharp  Pegar  Update()
       1 referência
26     void Pegar_()
27     {
28         RaycastHit hit;
29         if (Physics.Raycast(cam.transform.position, cam.transform.forward, out hit, 3f))
30         {
31             Debug.Log("Raio acertou: " + hit.collider.name);
32
33             if (hit.rigidbody != null)
34             {
35                 Debug.Log("Tem Rigidbody, pegando: " + hit.rigidbody.name);
36                 objeto = hit.rigidbody;
37                 objeto.isKinematic = true;
38                 objeto.GetComponent<Collider>().enabled = false;
39                 objeto.transform.SetParent(mao);
40                 objeto.transform.localPosition = Vector3.zero;
41             }
42             else
43             {
44                 Debug.Log("Acertou, mas não tem Rigidbody");
45             }
46         }
47         else
48         {
49             Debug.Log("O raio não acertou nada");
50         }
51     }
52
       1 referência
```

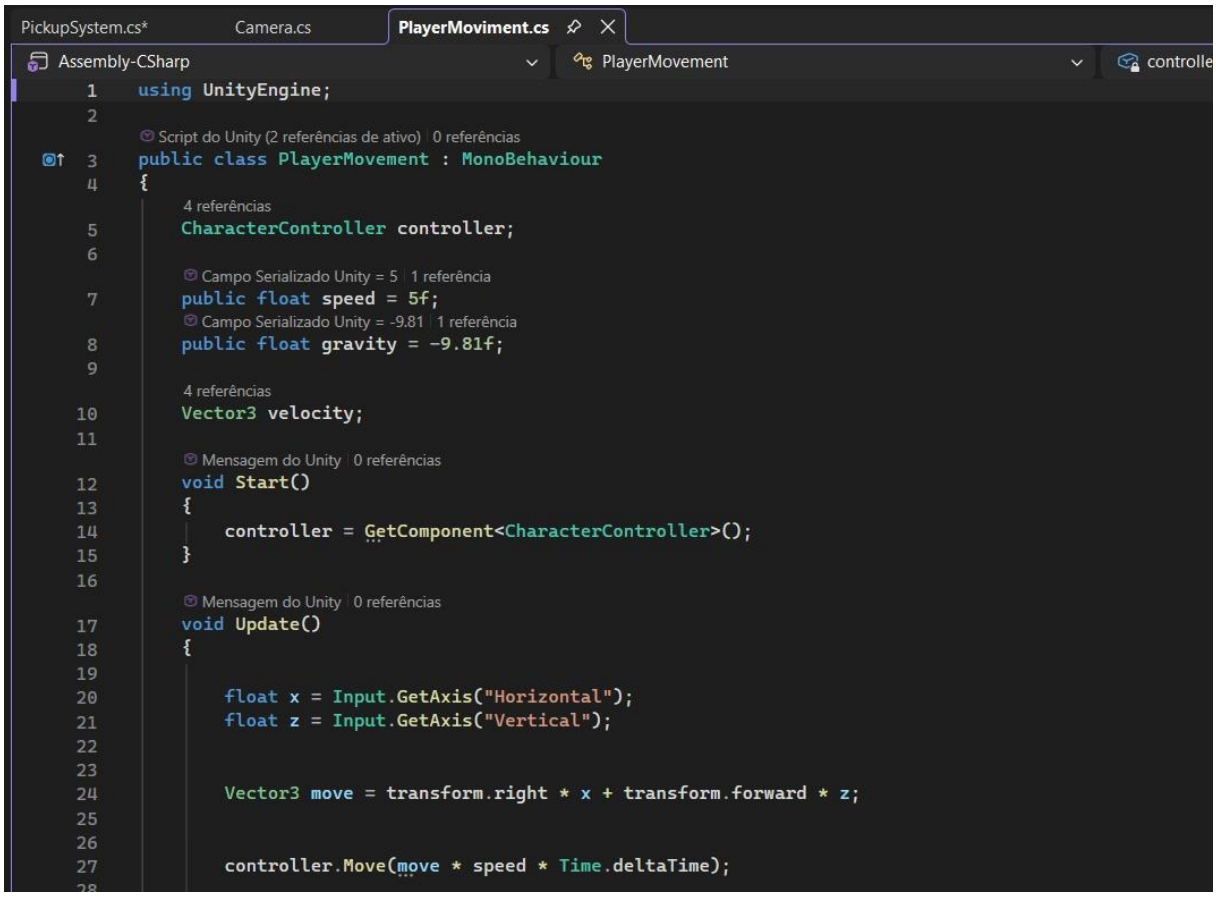
Fonte: Elaborado pelos autores (2026).

Figura 12 – Código-fonte da classe Pegar: método Soltar (linhas 52 a 61)



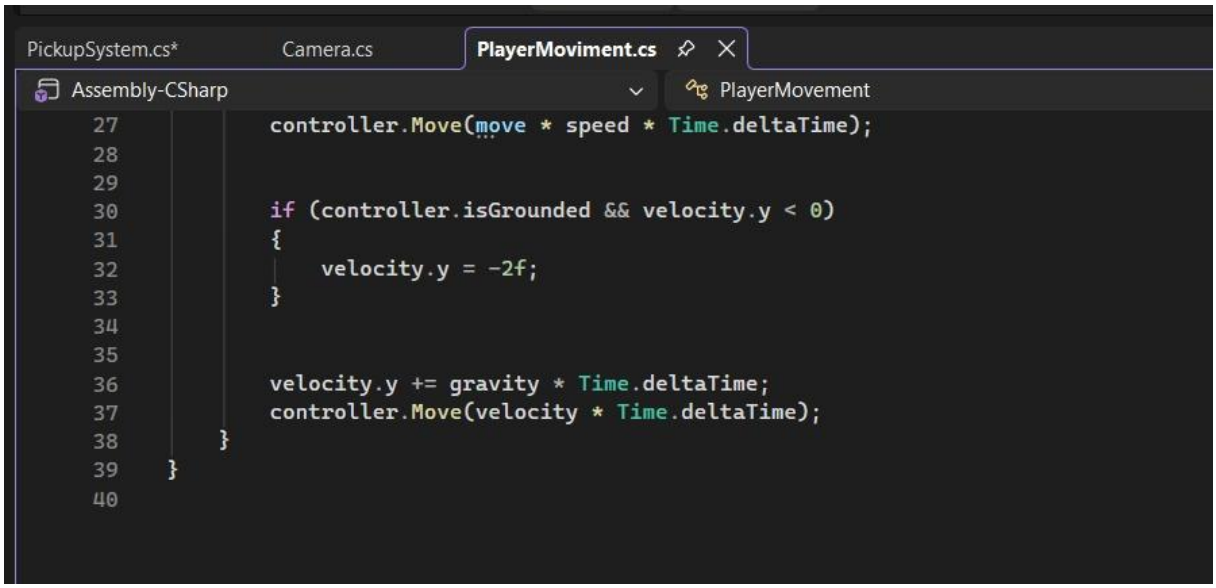
Fonte: Elaborado pelos autores (2026).

Figura 13 – Código-fonte de PlayerMovement.cs (linhas 1 a 28)



Fonte: Elaborado pelos autores (2026).

Figura 14 – Código-fonte de PlayerMovement.cs (linhas 27 a 40)



Fonte: Elaborado pelos autores (2026).